



Estruturas de Dados e Algoritmos 1

Aula 05

Ponteiros I

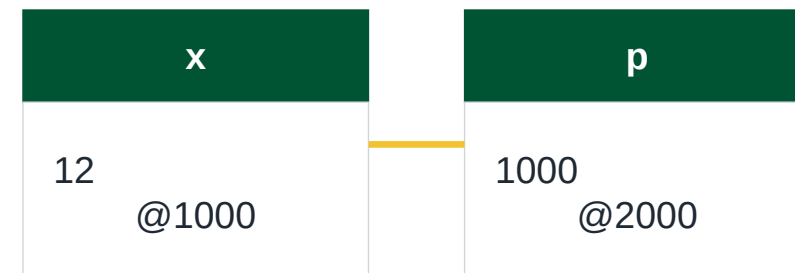
memória - endereço - operador & - indireção com *



**um ponteiro armazena
um endereço de memória**

Objetivos da aula

- Distinguir endereço de memória e conteúdo de uma variável.
- Declarar e inicializar ponteiros para int.
- Usar & para obter um endereço e * para acessar indiretamente um valor.
- Representar relações entre variáveis e ponteiros com diagramas de memória.
- Rastrear alterações feitas por meio de ponteiros.



**Hoje o foco é entender a relação
endereço -> ponteiro -> conteúdo.**

Motivação: como acessar outro dado indiretamente?

Na aula anterior:

```
int x = 10;  
int y = x;  
  
y = 25;
```

**Alterar y não altera x.
São variáveis diferentes.**

Pergunta para guiar a aula

**E se uma variável pudesse
guardar a localização de outra
variável na memória?**

Essa é a ideia central de um ponteiro.

Uma variável tem conteúdo e endereço

```
int x = 12;
```

variável x

conteúdo: 12
endereço: 1000

Conteúdo: valor armazenado

Endereço: localização na memória

Os endereços usados nos diagramas são simbólicos.
Em uma execução real, os valores podem ser diferentes.

Pergunta: se x está no endereço 1000, o que deveria significar guardar o valor 1000 em outra variável?

Operador &: obtendo o endereço

```
int x = 12;  
  
    &x    /* endereço de x */
```

x	
12	@1000

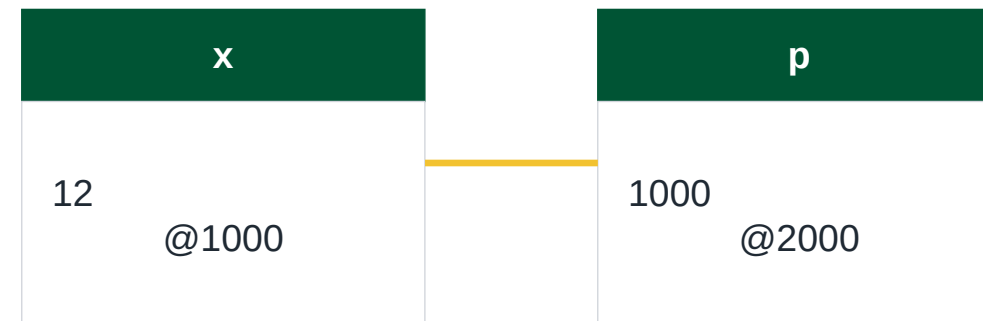
&x == 1000

Leia &x como: "o endereço da variável x".

& não lê o valor 12. Ele produz a localização onde x está armazenada.

Declarando e inicializando um ponteiro

```
int x = 12;  
int *p;  
  
p = &x;
```



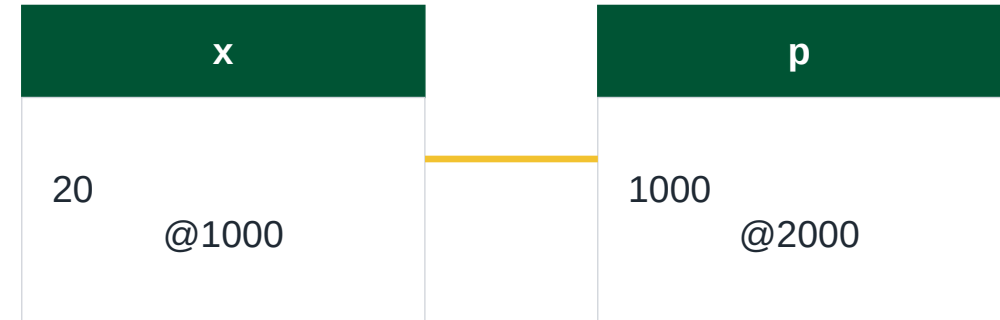
**p guarda o endereço de x.
Dizemos que p aponta para x.**

`int *p;`

p é uma variável cujo tipo é "ponteiro para int".

Operador *: acessando o valor apontado

```
int x = 12;  
int *p = &x;  
  
*p = 20;
```



***p** significa: "o valor armazenado no endereço guardado em p".

**Alterar *p altera x,
porque p aponta para x.**

O símbolo * aparece em dois contextos

1. Na declaração

```
int *p;
```

Declara p como ponteiro para int.

2. Em uma expressão

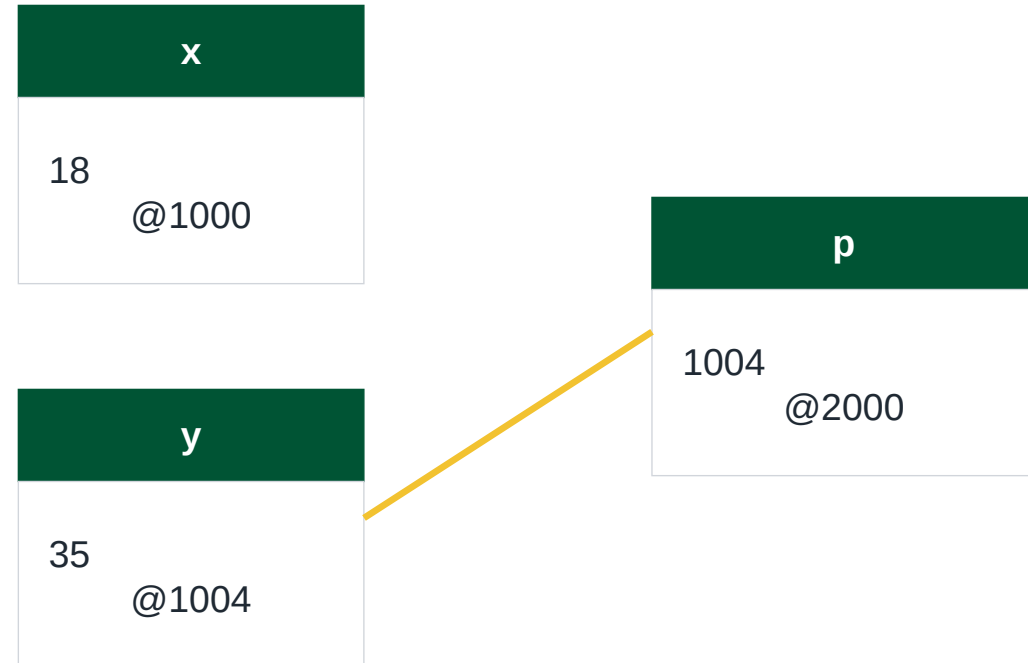
```
*p = 20;
```

Acessa o valor apontado por p.

A leitura depende do contexto: declaração de tipo versus indireção (desreferência).

Rastreamento: o ponteiro pode mudar de alvo

```
int x = 12;  
int y = 30;  
int *p = &x;  
  
*p = 18;  
p = &y;  
*p = *p + 5;
```

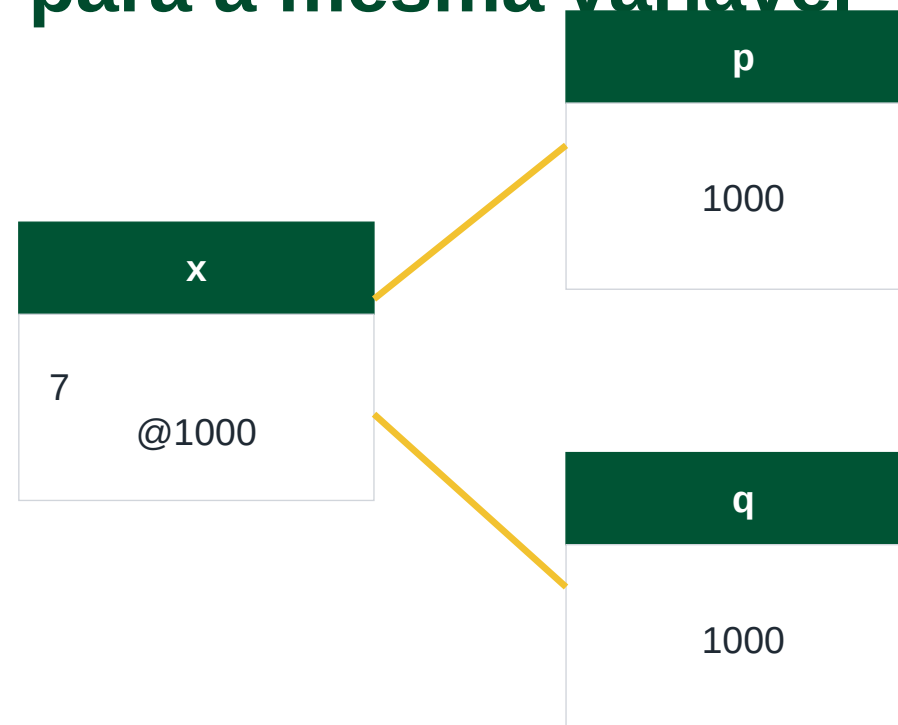


Estado final: p aponta para y.

Dois ponteiros podem apontar para a mesma variável

```
int x = 7;  
int *p = &x;  
      int *q = p;
```

**p e q são variáveis diferentes,
mas ambas apontam para x.**



Erro perigoso: usar um ponteiro não inicializado

```
int *p;  
  
*p = 10;    /* incorreto */
```

p ainda não recebeu um endereço válido.

Antes de usar *p, é preciso saber para onde p aponta.

Nesta aula, inicialize o ponteiro explicitamente com o endereço de uma variável compatível.

Miniatividade oral

```
int a = 4;  
int b = 9;  
int *p = &a;  
  
*p = b;  
p = &b;  
  
    *p = *p + 1;
```

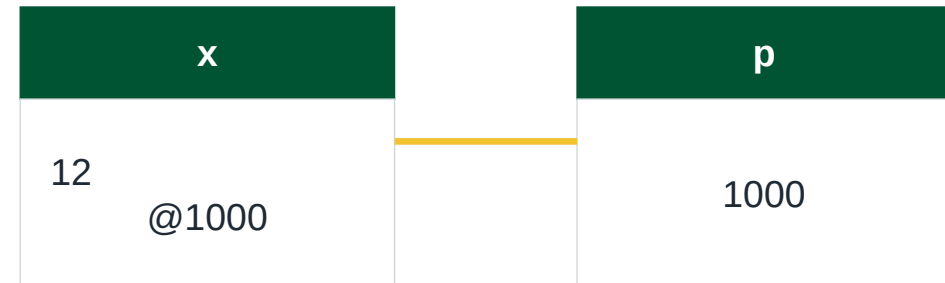
Responda sem compilador:

- Qual é o valor final de a?
- Qual é o valor final de b?
- Para qual variável p aponta ao final?
- Qual é o valor de *p?

WS02 - Ponteiros I

Durante o worksheet, os alunos devem:

- distinguir endereço e conteúdo;
- interpretar `p`, `&x` e `*p`;
- rastrear alterações de ponteiros;
- desenhar diagramas de memória;
- completar pequenas instruções em ANSI C.



**Estratégia: circule primeiro os endereços.
Depois acompanhe o alvo de cada ponteiro.**

Fechamento

Síntese

- Toda variável ocupa uma posição na memória.
- `&x` produz o endereço de `x`.
- `p` pode armazenar esse endereço.
- `*p` acessa o conteúdo apontado por `p`.
- Mudar `p` muda o alvo; mudar `*p` altera o alvo.

Próxima etapa

Ponteiros II: ponteiros como parâmetros, modificação indireta e relação com arrays.

A PC2 será encaminhada após esse aprofundamento.